

COMPUTER ARCHITECTURE

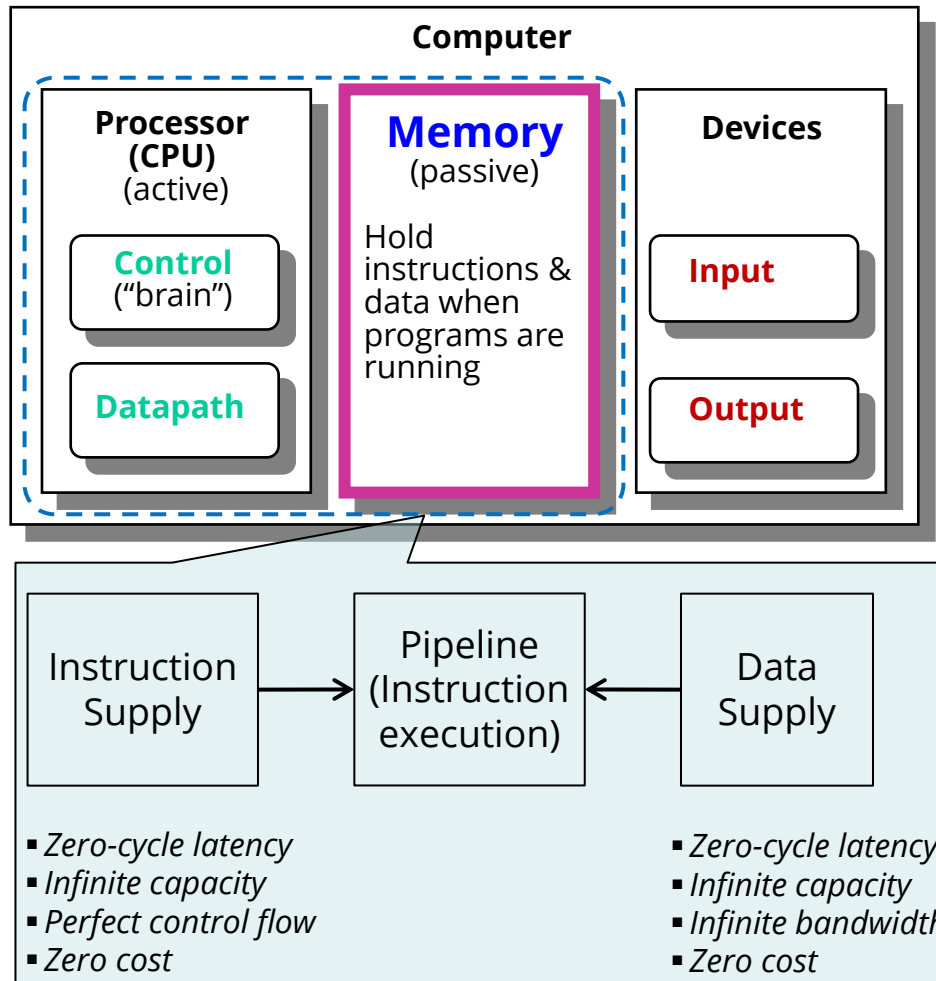


LECTURE 11: Memory hierarchy

CONTENTS

- > The problem with memory
 - The memory wall
- > The solution
 - Memory hierarchy
 - The memory locality principle
- > The implementation
 - Direct-mapped cache
 - Cache design considerations

The Ideal Memory (What We Want)



➤ From the programmer's perspective

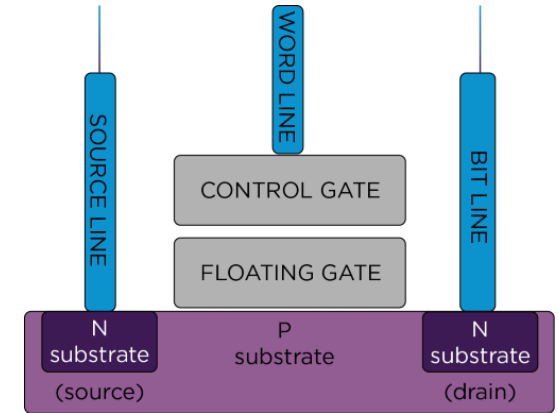
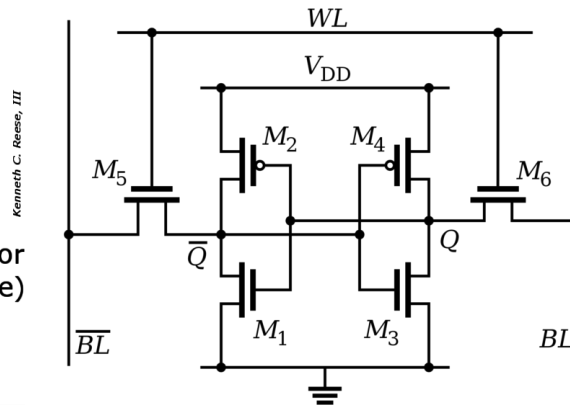
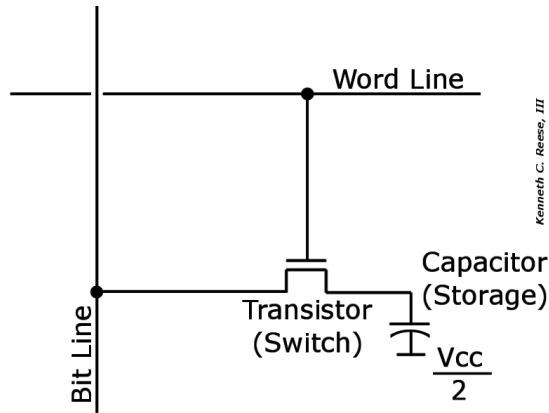
- Small physical memory appearing huge
- Slow physical memory appearing fast
- Transparent operation (hardware handles everything)

Life's easier for programmer, but creates major challenges for architects

➤ From the processor's perspective

- Zero-cycle latency (instant access)
- Infinite capacity (no size limits)
- Infinite bandwidth (unlimited throughput)
- Zero cost (free to build & operate!)

Memory Technology Landscape (What We Have)



DRAM (Dynamic RAM)

- High density
- Cheap
- Slow, leaky
- Destructive read

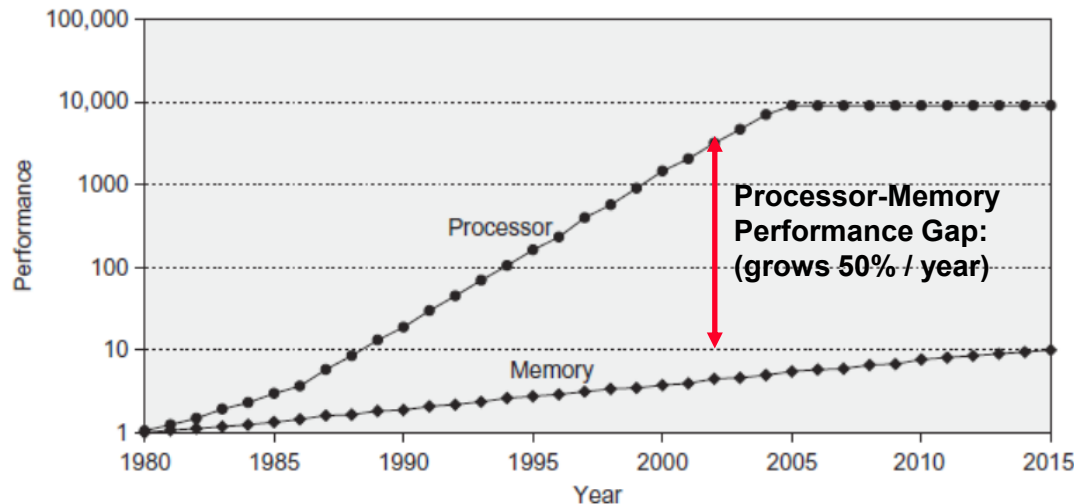
SRAM (Static RAM)

- Low density
- Expensive
- Very fast & stable
- Non-destructive read

Flash

- High density
- Cheaper than DRAM
- Slower than DRAM
- Non-volatile

The Memory Wall



- **Physical** memory size is limited
- Processor vs memory **speed disparity** continues to grow
- ⇒ Processor-Memory: an **unbalanced** system

Physics of Storage: You can't have it all

- Bigger is slower
 - Longer to determine the location
- Faster is more expensive
 - SRAM vs. DRAM vs. Disk vs. Tape
- Higher bandwidth is more expensive
 - More banks, more ports, ...



Memory Hierarchy Concept

- ❑ The problem: We want both fast and large memory
 - But we cannot achieve both with a single level of memory
- ❑ Architectural solution: Have **multiple levels of storage**
 - Progressively bigger and slower as the levels are farther from the processor
 - Top (closest to CPU): Small, expensive, ultra-fast
 - Bottom (furthest): Huge, cheap, slow
 - Ensure most of the data the processor needs is kept in the faster levels
 - Each level acts as a **cache** for the level below it
- ❑ Why does it work? Because programs are predictable
 - Loops repeat instruction sequences
 - Functions operate on local variables
 - Data structures stored contiguously
 - Sequential control flow dominates

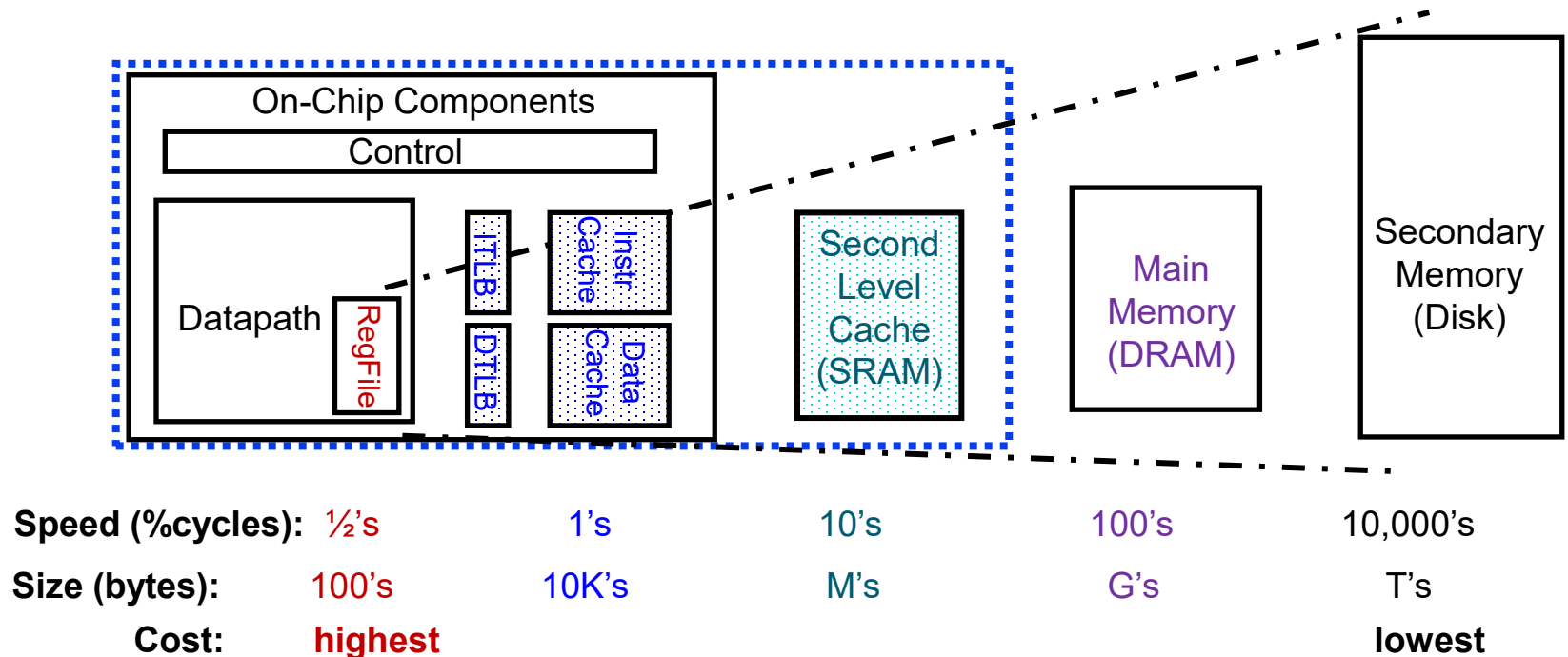
The Memory Locality Principle

- ❑ A “typical” program has a lot of **locality** in memory references
 - This is one of the most important principle in computer design.
- ❑ **Temporal** locality (locality in time)
 - Programs tend to reuse recently accessed data
 - Loop variables (reused every iteration)
 - Instructions in loops (executed repeatedly)
 - **Design response:** Keep recently used items in fast memory
- ❑ **Spatial** locality (locality in space)
 - Programs tend to access data near recently accessed data
 - Sequential instruction execution
 - Array traversals
 - Accessing structure fields
 - **Design response:** Fetch blocks of contiguous data, not just single words

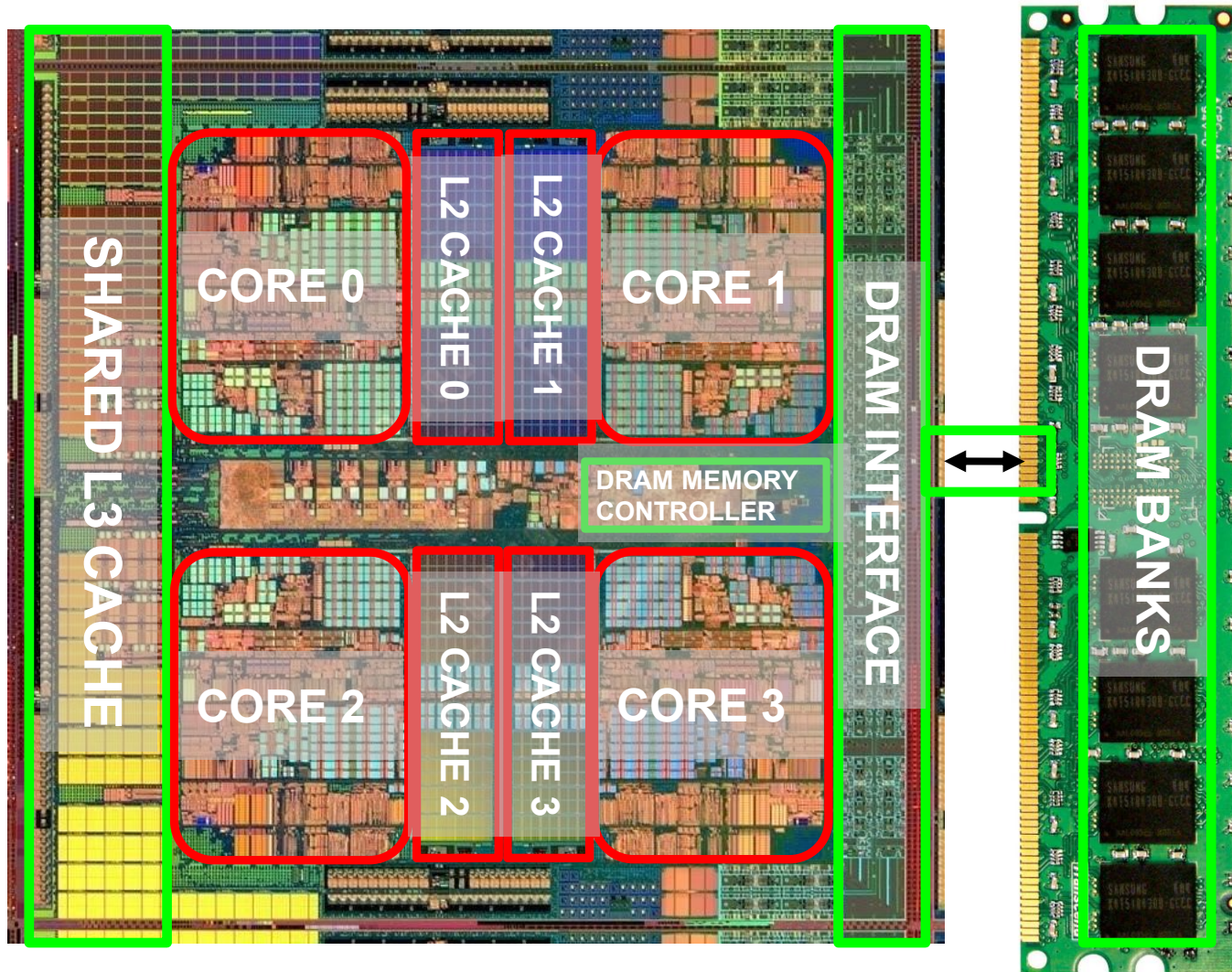
Implementation of the Memory Hierarchy

> A layered caching system

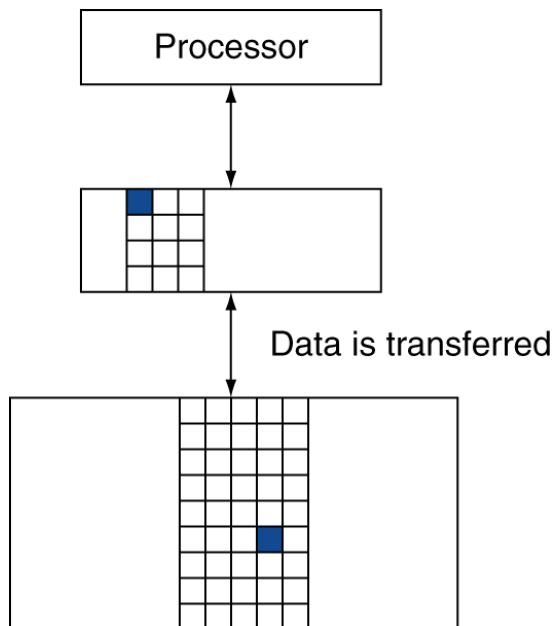
- Store everything on disk
- Copy recently accessed items from disk to smaller DRAM memory
- Copy more recently accessed items from DRAM to smaller SRAM memory



Real-life Example



Memory Hierarchy Characteristics



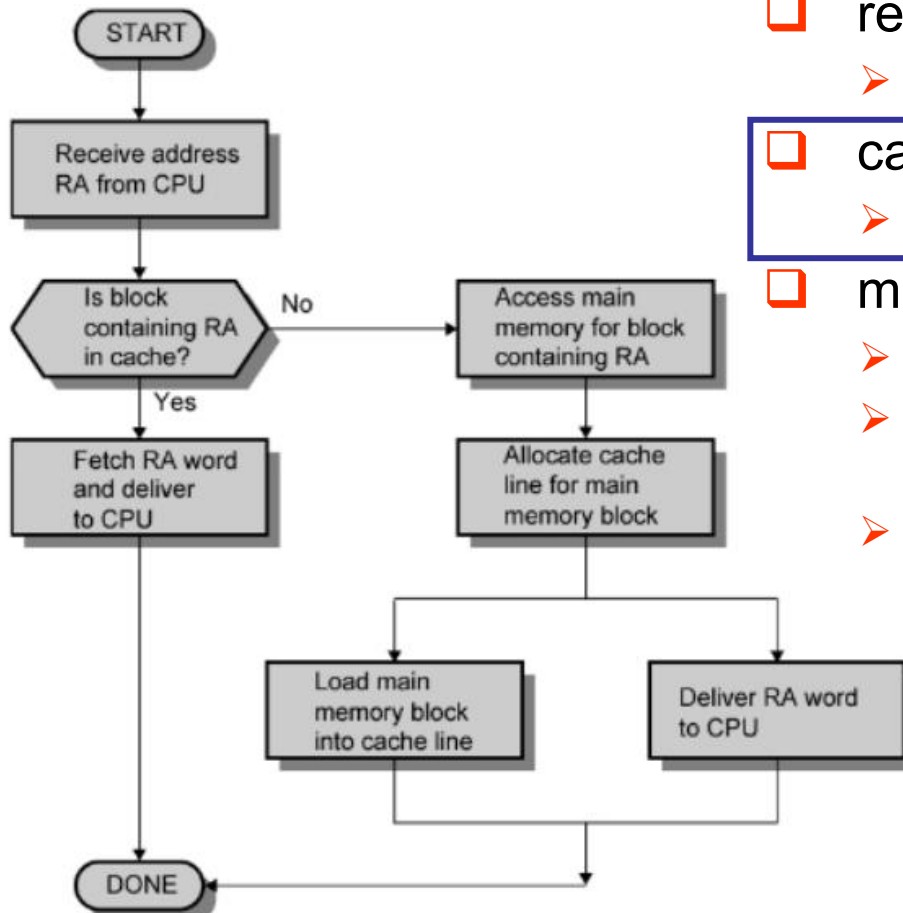
❑ Data organization:

- **Block (Line):** The minimum transfer unit between levels (usually multiple words)
- **Inclusion** property: Data in faster levels is typically a subset of slower levels

❑ Performance metrics

- **Hit:** Data found in the current level
 - **Hit rate** = $\frac{\text{\#hits}}{\text{\#accesses}}$
 - Hit Time = Access time + Hit/miss determination time
- **Miss:** Data not found → must fetch from lower level
 - **Miss rate** = $\frac{\text{\#miss}}{\text{\#accesses}} = 1 - (\text{Hit rate})$
 - Miss Penalty = Time to fetch block from lower level (typically >> hit time)

How is the Hierarchy Managed?



□ registers ↔ memory

➤ by compiler/programmer

□ cache ↔ main memory

➤ by the **cache controller hardware**

□ main memory ↔ disks

➤ by the operating system (virtual memory)

➤ virtual to physical address mapping assisted by the hardware (**TLB**)

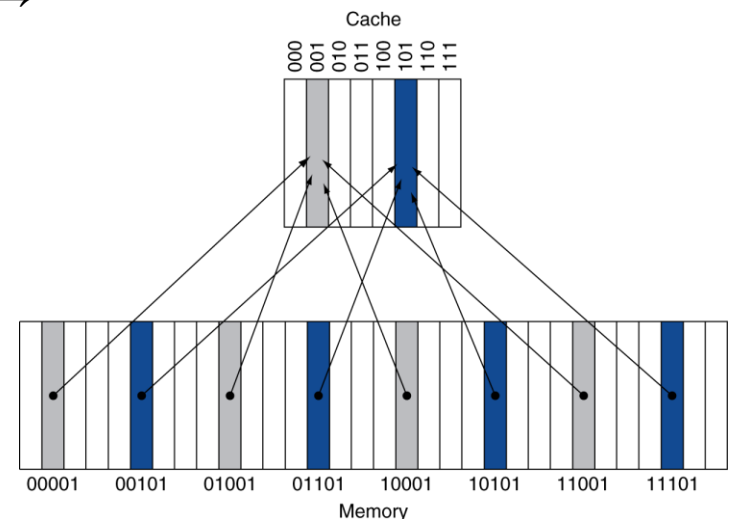
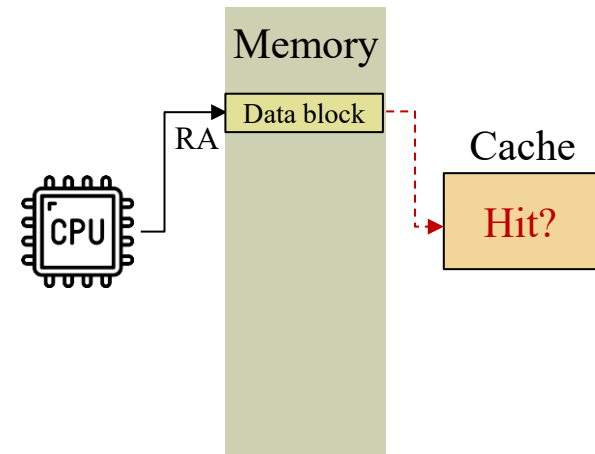
➤ by the programmer (files)

Cache Basics

- ❑ Two questions to answer:
 1. How do we know if a data is in the cache?
 2. If it is, how do we find it?
- ❑ Q2's simplest answer: **direct** mapped
 - Map memory address to cache location:
Location = Address **modulo** #Blocks
 - #Blocks in cache is usually a power of 2 →
Location $\equiv$ low-order address bits.

- ❑ **Example:** an 8-block cache

- The 3 least significant bits of the memory address index the cache location.
- Multiple data items in the memory can **share** the same block in the cache.



Tags and Valid Bits

- ❑ **[Q1]** How do we know if a requested word is in the cache?
 - Have a **tag** associated with each cache block that contains the address information (the upper portion of the address).
- ❑ What if there is no data in a location?
 - Add a **valid bit** to indicate that the associated block in the hierarchy contains valid data.
 - If valid bit = 0: the data is **not** valid → there cannot be a match for this block.

- ❑ **Example:** Consider the main memory word reference string

0 1 2 3 4 3 4 15

- Data memory allocation is given below

Word address	00 00	00 01	00 10	00 11	01 00	11 11
Data	0	1	2	3	4	15

- Start with an **empty** cache - all blocks initially marked as **not valid**.

Example Solution

Main memory

Word address	00 00	00 01	00 10	00 11	01 00	11 11
Data	0	1	2	3	4	15

Reference String : 0 1 2 3 4 3 4 15

8 requests, 6 misses

0 miss

Idx.	Val.	Tag	Data
00	1	00	Mem(0)

1 miss

Idx.	Val.	Tag	Data
00	1	00	Mem(0)
01	1	00	Mem(1)

2 miss

Idx.	Val.	Tag	Data
00	1	00	Mem(0)
01	1	00	Mem(1)
10	1	00	Mem(2)

3 miss

Idx.	Val.	Tag	Data
00	1	00	Mem(0)
01	1	00	Mem(1)
10	1	00	Mem(2)
11	1	00	Mem(3)

4 miss

Idx.	Val.	Tag	Data
00	1	00	Mem(0)
01	1	00	Mem(1)
10	1	00	Mem(2)
11	1	00	Mem(3)

3 hit

Idx.	Val.	Tag	Data
00	1	01	Mem(4)
01	1	00	Mem(1)
10	1	00	Mem(2)
11	1	00	Mem(3)

4 hit

Idx.	Val.	Tag	Data
00	1	01	Mem(4)
01	1	00	Mem(1)
10	1	00	Mem(2)
11	1	00	Mem(3)

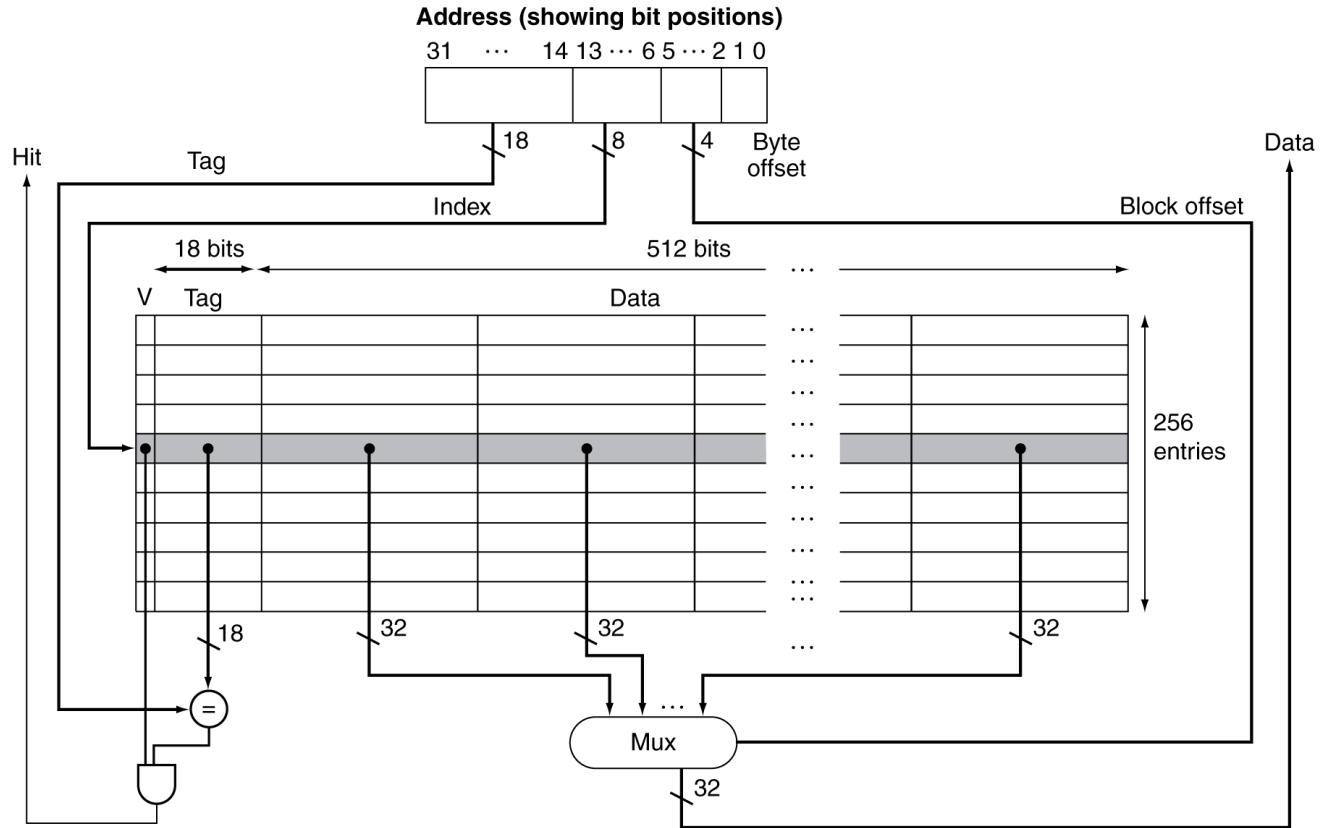
15 miss

Idx.	Val.	Tag	Data
00	1	01	Mem(4)
01	1	00	Mem(1)
10	1	00	Mem(2)
11	1	00	Mem(3)

11

15

Multiword Block Cache

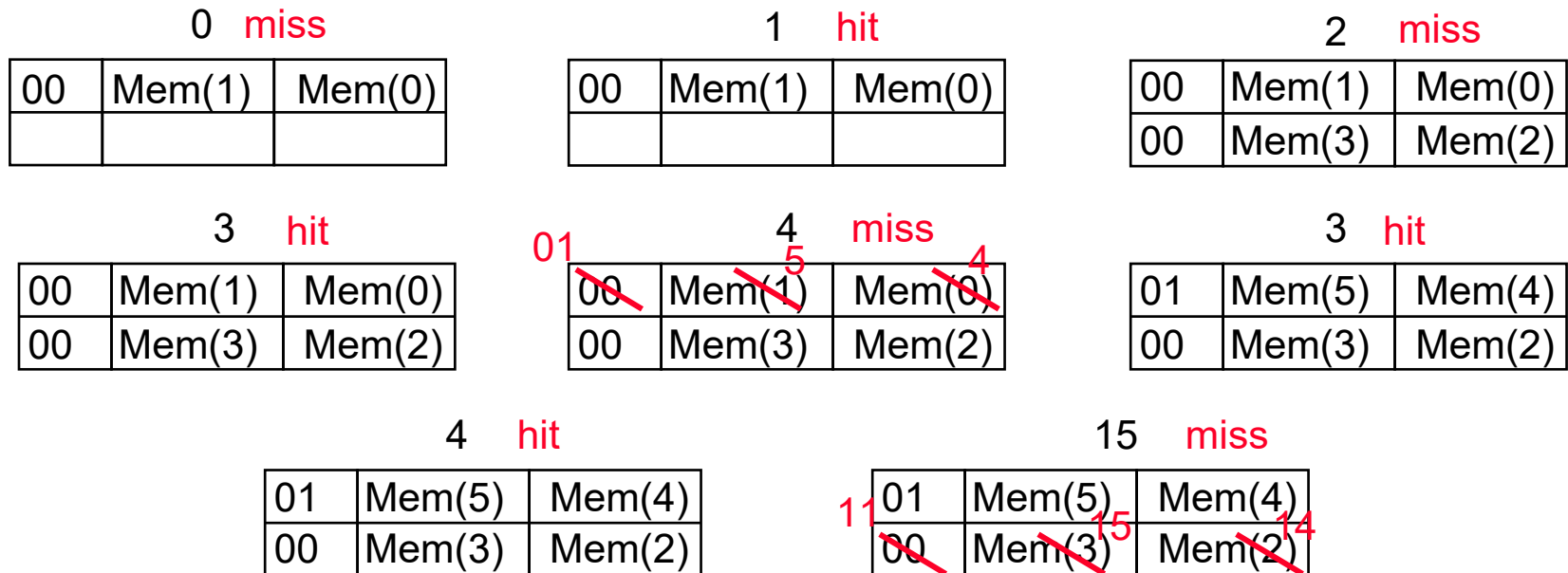


Multiword Block Cache Example

Retake the previous example with 2-word block cache now.

Word address	00 00	00 01	00 10	00 11	01 00	11 11
Data	0	1	2	3	4	15

Reference String : 0 1 2 3 4 3 4 15	8 requests, 4 misses
-------------------------------------	----------------------

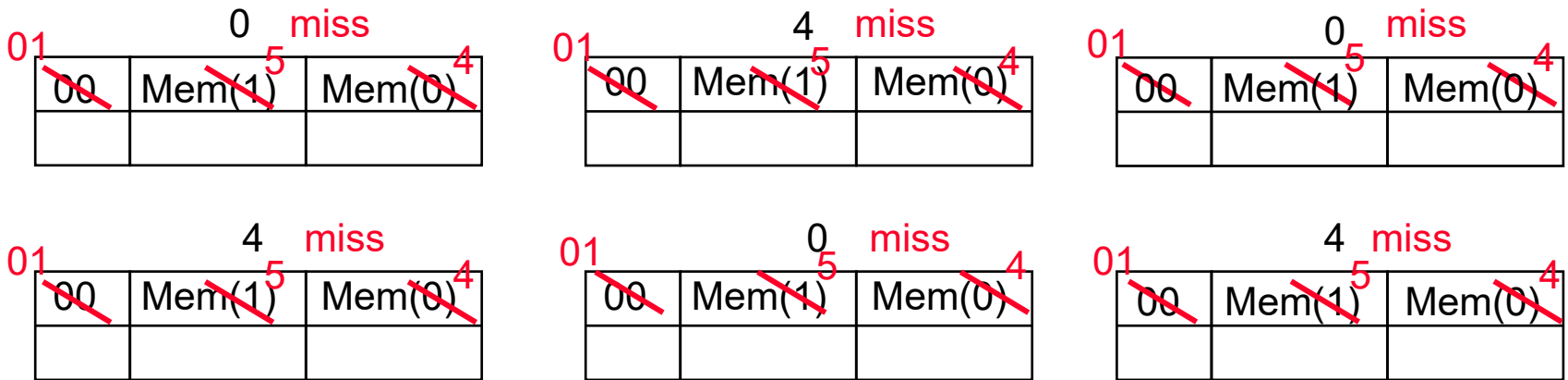


Conflict Miss in Direct Mapped Cache

❑ Consider the main memory word reference string:

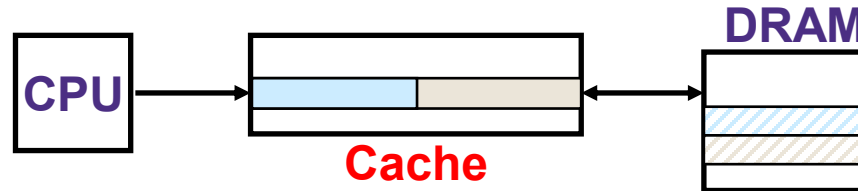
0 4 0 4 0 4 0 4

➤ Start with an empty cache - all blocks initially marked as not valid.



❑ Conflict misses arise from ping-pong references to two memory locations that map to the same cache block.

Cache Write Handling



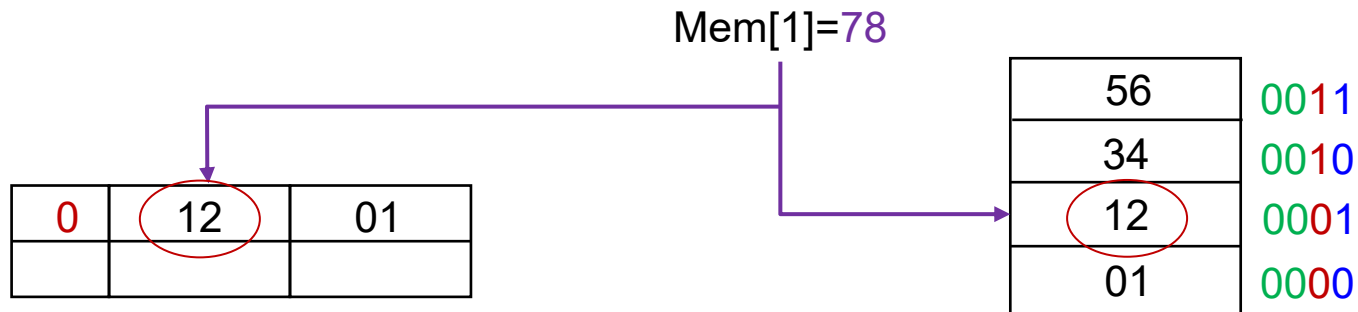
❑ Cache write policies

- Write hit: When the write address has already been cached
- Write miss: When the write address has **NOT** been cached

❑ Handling cache writes is more complex than reads

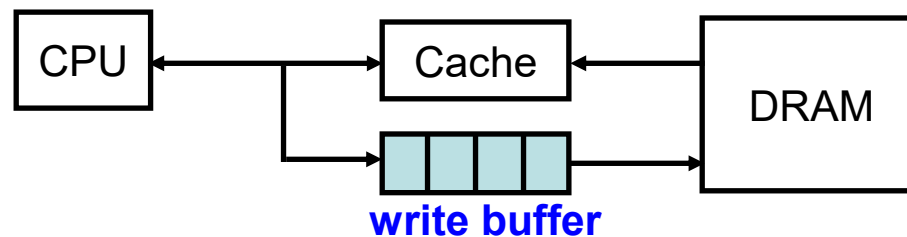
- Must maintain **consistency** between cache and main memory
- Requires performance trade-offs
 - Speed
 - Complexity
 - Coherence needs

Write Hit Policies



❑ Write through: write to both cache & memory

➤ Memory always up to date, but needs a write buffer to avoid stalling CPU

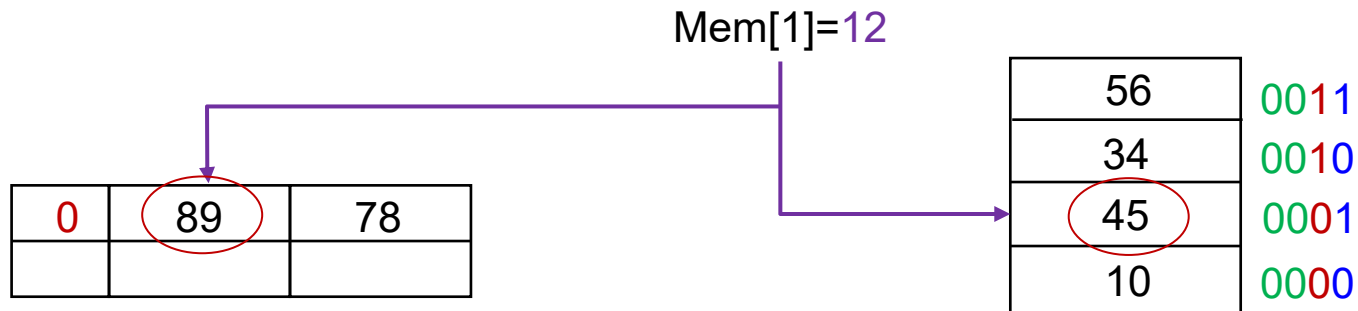


❑ Write-Back: write only to the cache, memory is updated later

➤ More complex: Requires a dirty bit to indicate cache block has been modified

➤ Low memory traffic: Only dirty blocks are written back when **evicting**

Write Miss Policies



❑ Write allocate (Fetch on Write)

- The block is first loaded into the cache, then write to the newly fetched block
- Future cache hits are likely but miss penalty increases (must fetch block first)
- Can evict other useful data if the data is not reused soon (cache pollution)

❑ No-write allocate

- Data is written directly to the memory, do **NOT** load block into cache.
- Simple logic, can avoid cache pollution
- Breaks locality: Subsequent reads to nearby locations will incur misses

Combined Cache Write Policies

- ❑ When a write miss occurs, the CPU's handling involves:
 - Determining whether to bring the block into the cache
 - Deciding when and how the data is written to the memory
- ❑ **Example 1:** write back + write allocate cache on a write miss
 - Block is fetched from memory and placed into cache
 - CPU writes new data to cache block, leaving the memory unchanged
 - A **dirty bit** associated with this block is set, indicating memory inconsistency
 - Later, when evicted, dirty block is written back to memory.
- ❑ **Example 2:** write through + no write allocate on a write miss
 - The data block is **not** fetched into the cache.
 - CPU writes new data to memory; the cache block remains unchanged.
 - Emphasis is on immediately updating memory, bypassing the cache.
 - Note: for write **hit**, CPU simultaneously writes to cache and memory.

Design Trade-offs: Cache Size

❑ Small cache

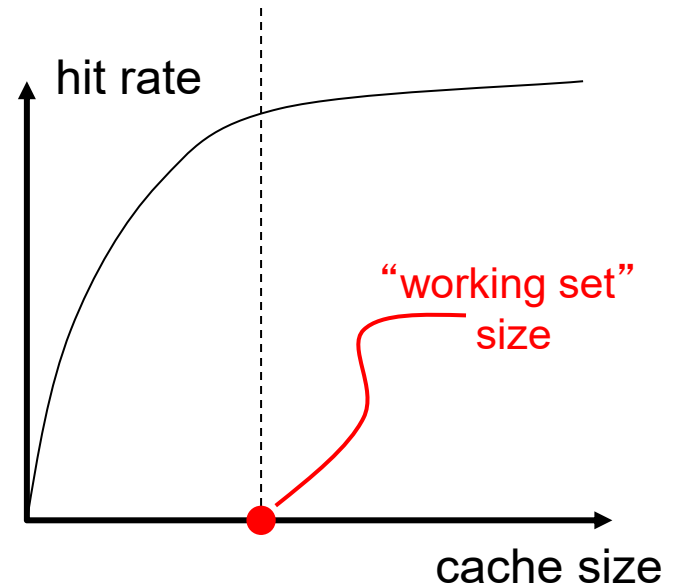
- doesn't exploit temporal locality well
→ increases miss rate
- useful data replaced often

❑ Large cache

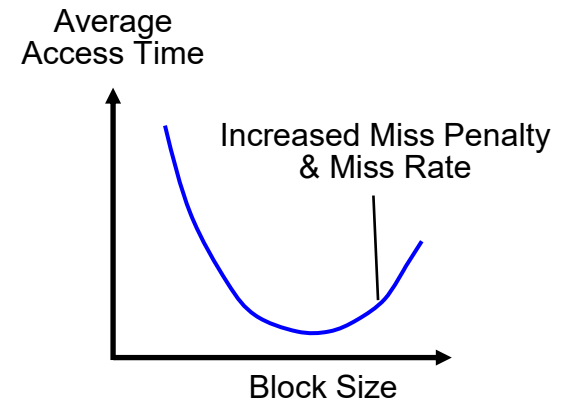
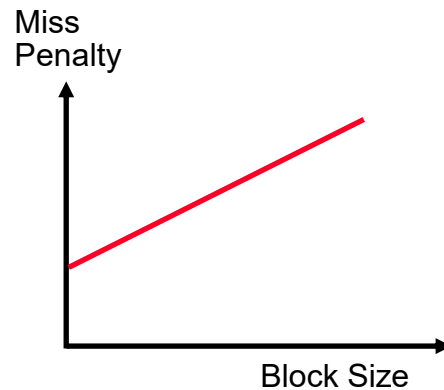
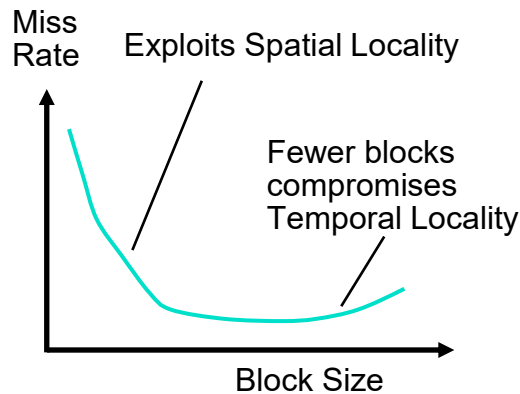
- can exploit temporal locality better → improves miss rate
- **not ALWAYS** better: bigger is slower

❑ Optimal size: The working set

- The subset of the total data that a running program is repeatedly accessing.
- A well-designed cache is large enough to contain the program's working set.



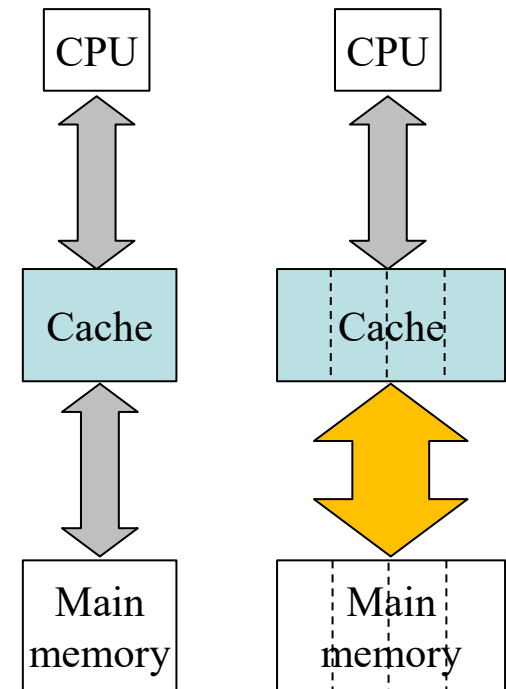
Design Trade-offs: Block Size



- ❑ Larger blocks should reduce miss rate (due to spatial locality)
 - But: larger blocks → fewer of them, more competition → increased miss rate.
- ❑ Larger block size means larger miss penalty
 - Bigger is slower → takes longer to transfer the block into the cache

Main Memory Supporting Caches

- ❑ Main memory can be organized to reduce miss penalty. Assume the following steps are taken on a cache block read:
 - 1 bus cycle for address transfer
 - 15 bus cycles per DRAM access
 - 1 bus cycle per data transfer
- ❑ For 1-word block, 1-word-wide DRAM:
 - Miss penalty: $1 + 15 + 1 = 17$ bus cycles
- ❑ For 4-word block, 1-word-wide DRAM:
 - Miss penalty = $1 + 4 \times 15 + 4 \times 1 = 65$ bus cycles
- ❑ For 4-word block, 4-word-wide DRAM:
 - Miss penalty = $1 + 15 + 1 = 17$ bus cycles
 - Disadvantages: increased cost for wider bus



Writing Cache Friendly Code

- ❑ Two major rules
 - Repeated references are good (temporal locality)
 - Stride-1 reference patterns are good (spatial locality)
- ❑ Example: cold cache, 4-byte word, 4-word cache blocks

```
int sum_array_rows(int a[M][N])
{
    int i, j, sum = 0;

    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            sum += a[i][j];
    return sum;
}
```

Miss rate = $1/4 = 25\%$

```
int sum_array_cols(int a[M][N])
{
    int i, j, sum = 0;

    for (j = 0; j < N; j++)
        for (i = 0; i < M; i++)
            sum += a[i][j];
    return sum;
}
```

Miss rate = 100%

CONCLUSION

Key takeaways

- > Memory hierarchy exploits the principle of locality
 - Temporal Locality
 - Spatial Locality
- > Cache operations
 - Hits/misses
 - Write policies
- > Main memory can be organized to help caching
 - Wider bus, interleaved memory

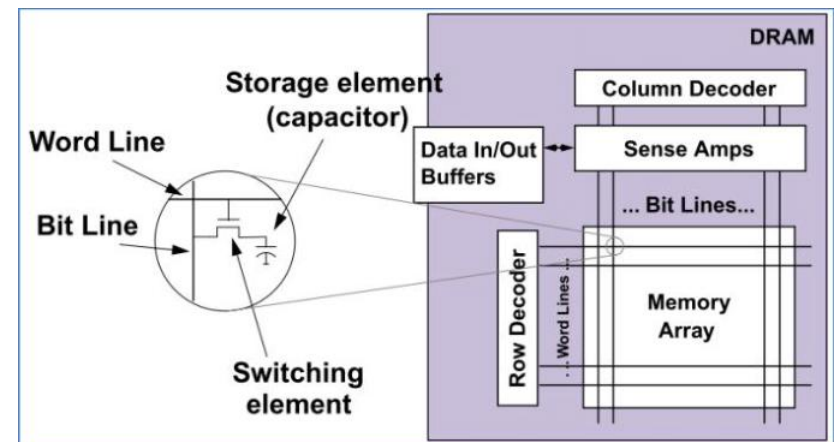
NEXT LECTURE

[Flipped class] Memory hierarchy (cont.)

- > Pre-class
 - Homework
 - Self study
- > In class
 - Reinforcement/enrichment discussion
 - Quizzes
- > Post class
 - Homework
 - Consultation (if needed)

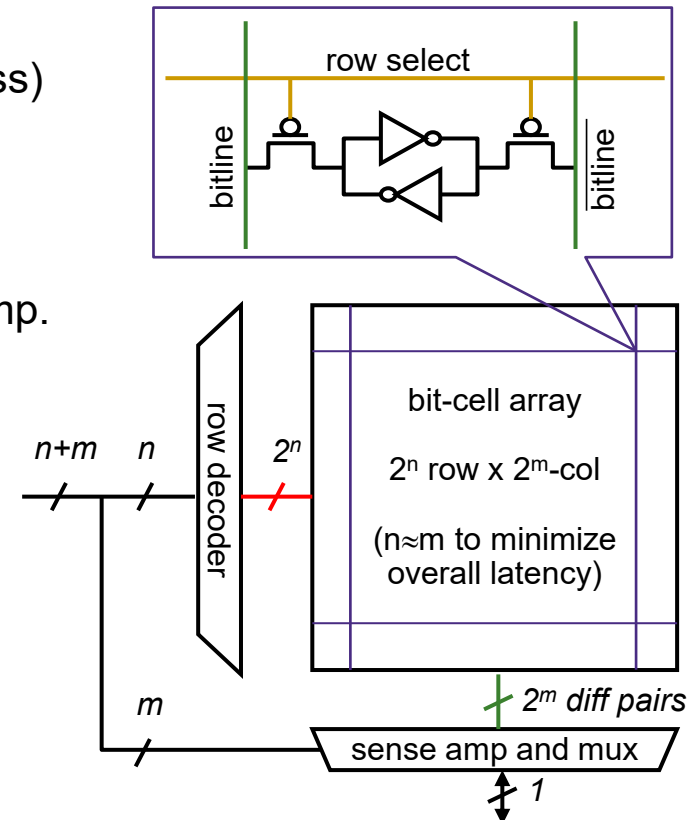
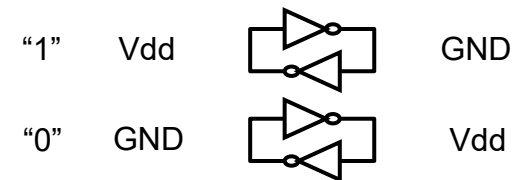
Dynamic Random Access Memory (DRAM)

- ❑ Used for: Main memory.
- ❑ Structure: 1 capacitor + 1 transistor per bit
 - Capacitor charge state represents 0 or 1
 - Access transistor (FET) controls read/write
- ❑ Operations:
 - Write: Turn on FET, charge/discharge capacitor via bit lines
 - Read: Turn on FET, sense tiny voltage change (destructive!)
- ❑ Characteristics:
 - **Dense**: 1 transistor/bit = high capacity
 - **Cheap**: simple structure, low cost/bit
 - **Destructive read**: Data must be rewritten after read
 - **Leakage**: Capacitors lose charge → requires periodic refresh

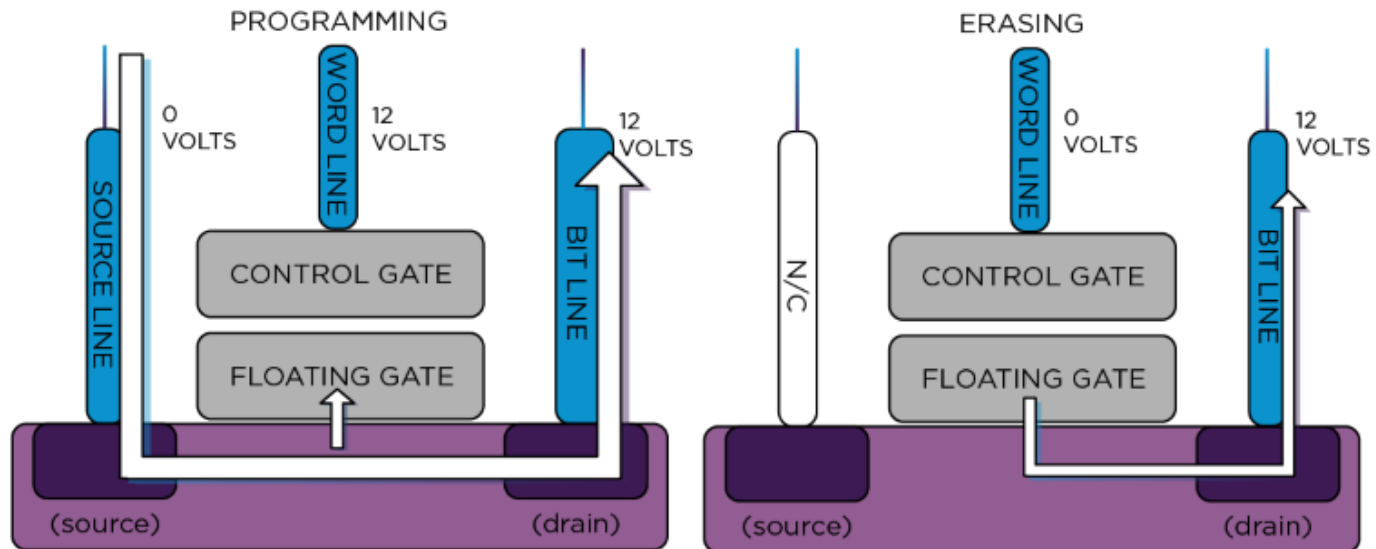


Static Random Access Memory (SRAM)

- ❑ Used for: Caches (L1, L2, L3).
- ❑ Structure: a bistable latch
 - Stable states represent 0 and 1
 - 6 transistors per bit (4 for storage, 2 for access)
- ❑ Operation:
 - Write: Overwrite the latch state via bit lines
 - Read: Connect the latch to bit lines; sense amp. detects the stored value (non-destructive)
- ❑ Characteristics:
 - **Fast:** No cap. charge/discharge delay
 - **Simple read:** Non-destructive
 - **No refresh needed:** No cap. leakage
 - **Expensive:** 6 transistors/bit → larger area
 - **Lower density:** Smaller capacity



Flash Memory



- > Used for: Solid-state drives (SSD), non-volatile storage
 - **Very dense:** multiple bits/transistor, read/written in blocks
 - **Slower** than DRAM (especially on writes)
 - **Limited number of writes:** charging/discharging the floating gate requires large voltages that damage transistor
 - Long time technology of choice for non-volatile storage: higher-performance but higher-cost replacement for HDD.